МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ

Федеральное государственное автономное образовательное учреждение

высшего образования

«СЕВЕРО-КАВКАЗСКИЙ ФЕДЕРАЛЬНЫЙ УНИВЕРСИТЕТ»

Институт перспективной инженерии

Департамент цифровых, робототехнических систем и электроники

Отчет к лабораторной работе № 2

Тема: Порождающие паттерны: Абстрактная фабрика, Фабричный метод, Прототип

Дисциплина: **«Паттерны проектирования»**

Выполнил:

Студент группы ПИЖ-б-о-23-1
направление подготовки
09.03.04 «Программная инженерия»

Постников Илья Викторович

Проверил:

Доцент ДЦРСИЭ

Альбекова З. М.

Ставрополь 2026

Цель: изучить принцип работы порождающих паттернов проектирования «Абстрактная фабрика» (Abstract Factory) и «Прототип» (Prototype) и реализовать прикладную задачу «Графический пароль» с использованием данных паттернов.

Задание: необходимо разработать приложение для работы с графическим паролем и реализовать два паттерна из списка: «Абстрактная фабрика», «Фабричный метод», «Прототип».

В соответствии с заданием были выбраны паттерны **«Абстрактная фабрика» (Abstract Factory)** и **«Прототип» (Prototype)**.

Вариант 8 — Графический пароль.

Разработанное приложение позволяет работать с последовательностями графического пароля, использовать готовые шаблоны и создавать их копии.

Для создания элементов графического интерфейса используются различные фабрики, позволяющие изменять оформление приложения.

1. Описание паттерна «Абстрактная фабрика»

Паттерн **«Абстрактная фабрика» (Abstract Factory)** является порождающим паттерном проектирования, который предоставляет интерфейс для создания семейств связанных объектов без указания их конкретных классов.

Данный паттерн используется в ситуациях, когда приложение должно работать с несколькими группами взаимосвязанных объектов, но конкретные классы этих объектов должны определяться отдельно.

В разработанном приложении паттерн «Абстрактная фабрика» используется для создания элементов графического интерфейса приложения «Графический пароль».

В программе существует абстрактная фабрика PasswordFactory, которая определяет методы создания элементов графического пароля.

На её основе реализованы две конкретные фабрики:

- DarkPasswordFactory — создаёт элементы для тёмной темы;
- LightPasswordFactory — создаёт элементы для светлой темы.

Обе фабрики предоставляют одинаковый интерфейс, но создают элементы с различным оформлением.

Таким образом, основное приложение может использовать фабрику, не обращаясь непосредственно к конкретным классам создаваемых элементов.

2. Описание паттерна «Прототип»

Паттерн **«Прототип» (Prototype)** является порождающим паттерном проектирования, который позволяет создавать новые объекты путём копирования уже существующего объекта-прототипа.

Основная идея паттерна заключается в том, что вместо создания нового объекта с нуля можно создать копию существующего объекта и при необходимости изменить её свойства.

В разработанном приложении паттерн «Прототип» используется для работы с шаблонами графического пароля.

Для хранения шаблона используется класс PasswordPattern, содержащий название шаблона и последовательность выбранных ячеек.

В классе реализован метод clone(), который создаёт копию существующего шаблона.

В приложении используются готовые шаблоны графического пароля, например:

- «Крест»;
- «Квадрат»;
- «Зигзаг».

Пользователь может выбрать существующий шаблон и создать его копию. Полученная копия становится отдельным объектом и может использоваться независимо от исходного шаблона.

Таким образом, класс PasswordPattern выполняет роль прототипа, а метод clone() используется для создания его копий.

3. Описание разработанного приложения

Разработанное приложение предназначено для работы с графическими паролями и их шаблонами.

Приложение реализовано на языке **Python** с использованием библиотеки **Tkinter** для создания графического интерфейса пользователя.

Основное окно приложения содержит сетку из девяти ячеек, пронумерованных от 1 до 9. Пользователь может выбирать ячейки и формировать последовательность графического пароля.

Приложение позволяет:

- выбирать отдельные ячейки;
- формировать последовательность графического пароля;
- очищать выбранную последовательность;
- выбирать готовый шаблон;
- применять выбранный шаблон;
- создавать копию выбранного шаблона (кнопка «Клонировать шаблон»);
- переключаться между тёмной и светлой темами.

В программе используются три готовых шаблона:

1. **Крест**;
2. **Квадрат**;
3. **Зигзаг**.

При выборе шаблона появляется диалог подтверждения «Применить шаблон … ?». При подтверждении последовательность применяется к области графического пароля; при отказе шаблон остаётся выбранным.

Для изменения оформления приложения используются две конкретные фабрики: DarkPasswordFactory и LightPasswordFactory.

4. Диаграмма классов

Для отображения структуры разработанного приложения была создана UML-диаграмма классов.

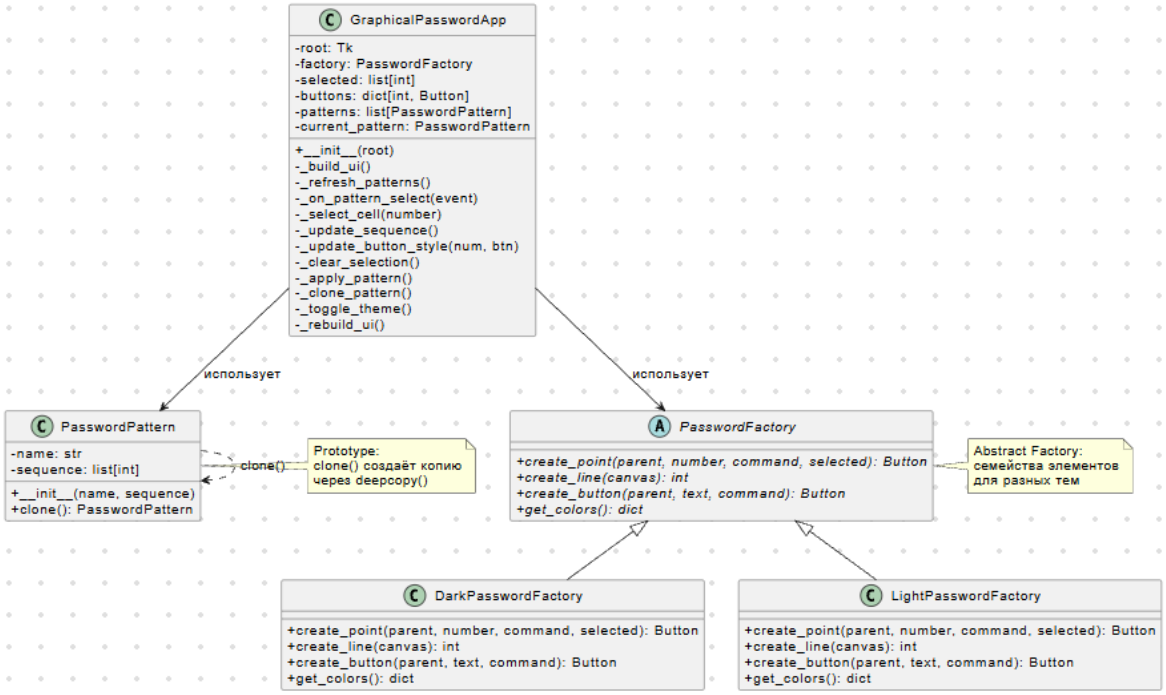


Рисунок 1 — Диаграмма классов приложения «Графический пароль»

Диаграмма содержит основные классы и интерфейсы приложения: GraphicalPasswordApp, PasswordFactory, DarkPasswordFactory, LightPasswordFactory и PasswordPattern.

Интерфейс PasswordFactory определяет общий набор методов для создания элементов графического интерфейса.

Классы DarkPasswordFactory и LightPasswordFactory реализуют данный интерфейс и отвечают за создание элементов для соответствующих тем оформления.

Класс PasswordPattern используется для хранения шаблонов графического пароля и реализует принцип паттерна «Прототип».

Класс GraphicalPasswordApp является основным классом приложения и использует фабрику для создания элементов интерфейса и объекты PasswordPattern для работы с шаблонами.

5. Реализация паттерна «Абстрактная фабрика»

Основная часть реализации паттерна «Абстрактная фабрика» находится в классе PasswordFactory.

В программе он представлен абстрактным классом:

```
class PasswordFactory(ABC):

    @abstractmethod
    def create_point(self, parent, number, command, selected=False):
        pass

    @abstractmethod
    def create_line(self, canvas):
        pass

    @abstractmethod
    def create_button(self, parent, text, command):
        pass

    @abstractmethod
    def get_colors(self):
        pass
```

Данный класс определяет общий интерфейс для создания элементов графического пароля.

На его основе реализованы две конкретные фабрики.

Тёмная фабрика:

```
class DarkPasswordFactory(PasswordFactory):

    def create_point(self, parent, number, command):
        return tk.Button(
            parent,
```

```
            text=str(number),
            ...
        )
```

Светлая фабрика:

```
class LightPasswordFactory(PasswordFactory):

    def create_point(self, parent, number, command):
        return tk.Button(
            parent,
            text=str(number),
            ...
        )
```

Основное отличие заключается в оформлении создаваемых элементов.

При переключении темы приложение меняет используемую фабрику:

```
if isinstance(self.factory, DarkPasswordFactory):
    self.factory = LightPasswordFactory()
else:
    self.factory = DarkPasswordFactory()
```

Таким образом, основной класс приложения не должен самостоятельно создавать элементы каждой конкретной темы. Для этого он обращается к выбранной фабрике.

6. Реализация паттерна «Прототип»

Основная часть реализации паттерна «Прототип» находится в классе PasswordPattern.

Класс содержит название шаблона и последовательность ячеек:

```
class PasswordPattern:

    def __init__(self, name, sequence):
        self.name = name
        self.sequence = sequence
```

Для создания копии объекта используется метод clone():

```
def clone(self):
    return deepcopy(self)
```

В программе определены готовые шаблоны:

```
CROSS_PATTERN = PasswordPattern(
    "Крест",
    [2, 5, 8, 4, 6]
)

SQUARE_PATTERN = PasswordPattern(
    "Квадрат",
    [1, 2, 3, 6, 9, 8, 7, 4]
)

ZIGZAG_PATTERN = PasswordPattern(
    "Зигзаг",
    [1, 2, 4, 3, 5, 7, 6, 8, 9]
)
```

При выборе шаблона пользователь может создать его копию:

```
cloned = self.current_pattern.clone()
```

После этого создаётся отдельный объект, который содержит те же данные, что и исходный шаблон.

Таким образом, паттерн «Прототип» позволяет создавать новые шаблоны на основе уже существующих объектов.

7. Результаты работы программы

После запуска приложения пользователю отображается главное окно программы с сеткой из девяти ячеек.

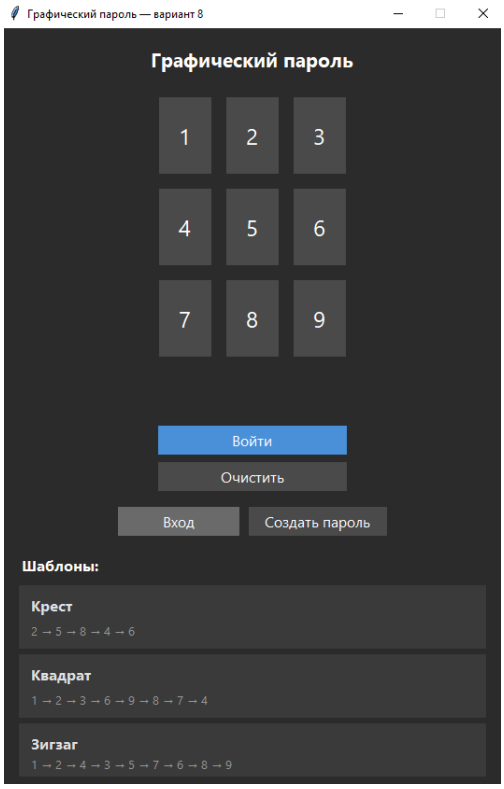


Рисунок 2 — Главное окно приложения

Пользователь может выбирать ячейки и формировать последовательность графического пароля.

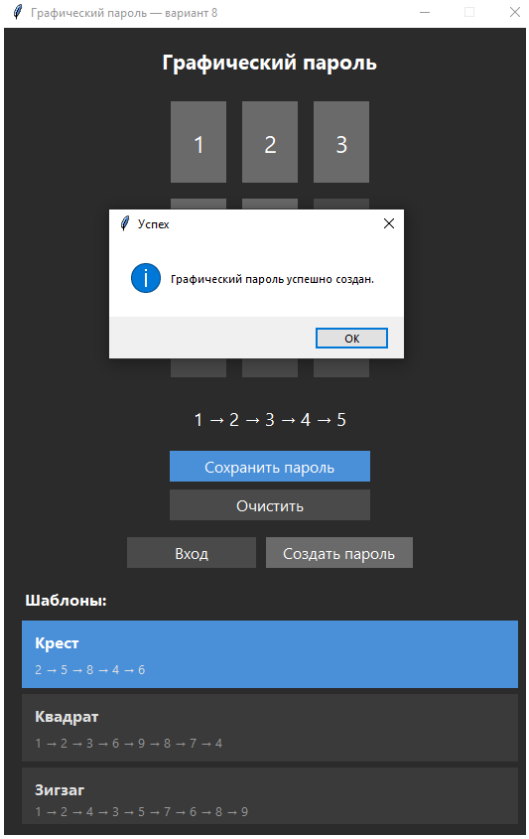


Рисунок 3 – Формирование последовательности графического пароля

В приложении также предусмотрен раздел с готовыми шаблонами графического пароля.

Пользователь может выбрать один из доступных шаблонов:

- «Крест»;
- «Квадрат»;
- «Зигзаг».

При клике на карточку шаблона появляется диалоговое окно с вопросом: «Применить шаблон … ?». При подтверждении последовательность применяется к области графического пароля и появляется уведомление «Использован шаблон: …». При отказе шаблон остаётся выбранным (подсвеченным), и его можно клонировать кнопкой «Клонировать шаблон» или применить позже кнопкой «Применить шаблон».

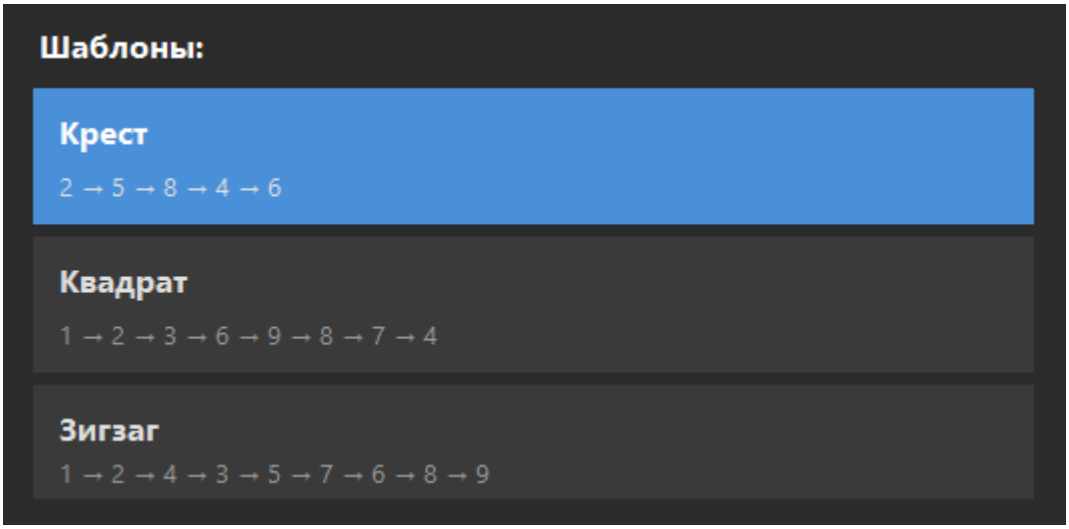


Рисунок 4 — Выбор шаблона графического пароля

После нажатия кнопки «Применить шаблон» соответствующая последовательность отображается в области графического пароля.

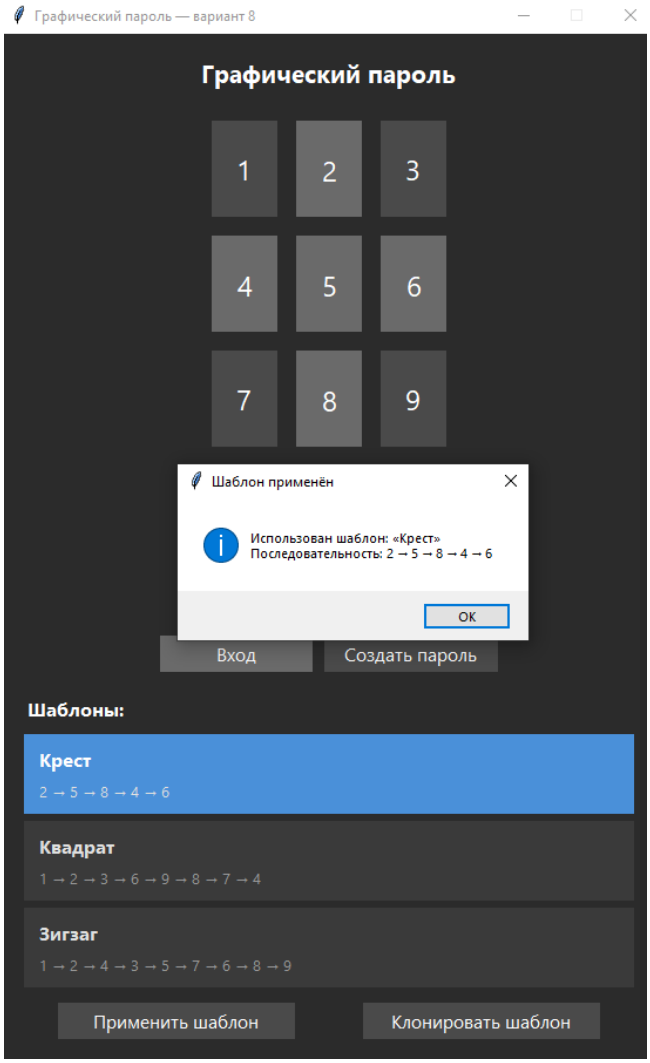


Рисунок 5 — Применение шаблона

Для демонстрации паттерна «Прототип» пользователь может выбрать шаблон (кликнуть по карточке, при отказе от применения шаблон остаётся выбранным) и нажать кнопку «Клонировать шаблон». После этого создаётся копия выбранного шаблона, которая добавляется в список шаблонов, и появляется уведомление: «Создана копия шаблона: «Квадрат»».

Например:

Квадрат

Квадрат — копия

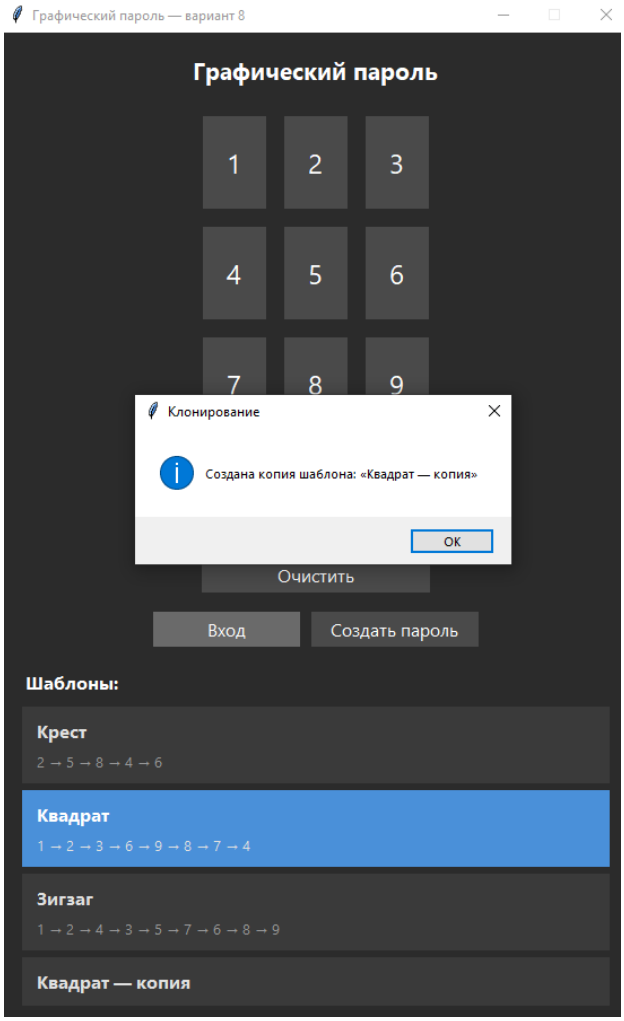


Рисунок 6 — Клонирование шаблона с использованием паттерна «Прототип»

8. Исходный код

Приложение разработано на языке программирования **Python**. Для создания графического интерфейса используется библиотека **Tkinter**.

Основные элементы программы:

- GraphicalPasswordApp — основной класс графического интерфейса;
- PasswordFactory — абстрактная фабрика;
- DarkPasswordFactory — фабрика элементов тёмной темы;
- LightPasswordFactory — фабрика элементов светлой темы;
- PasswordPattern — класс шаблона графического пароля и прототип;
- clone() — метод создания копии существующего шаблона;
- CROSS_PATTERN — шаблон «Крест»;

- SQUARE_PATTERN — шаблон «Квадрат»;
- ZIGZAG_PATTERN — шаблон «Зигзаг».

Полный исходный код программы представлен в приложении к лабораторной работе. Код доступен в репозитории GitHub: https://github.com/q1wf3/Patterny_Postnikov

Вывод

В ходе выполнения лабораторной работы было разработано приложение «Графический пароль» на языке программирования Python.

В процессе разработки были изучены и реализованы два порождающих паттерна проектирования: **«Абстрактная фабрика» (Abstract Factory)** и **«Прототип» (Prototype)**.

Паттерн «Абстрактная фабрика» был применён для создания элементов графического интерфейса приложения. В программе реализованы фабрики

DarkPasswordFactory и LightPasswordFactory, позволяющие использовать различные варианты оформления приложения.

Паттерн «Прототип» был применён для работы с шаблонами графического пароля. С помощью метода clone() пользователь может создавать копии существующих шаблонов без необходимости создавать их с нуля.

В результате была получена работоспособная программа с графическим интерфейсом, позволяющая формировать последовательности графического пароля, использовать готовые шаблоны, создавать их копии и переключать оформление приложения.

Таким образом, в ходе лабораторной работы были получены практические навыки применения порождающих паттернов **«Абстрактная фабрика»** и **«Прототип»** при разработке прикладного программного обеспечения.